

AI Driver Drowsiness & Distraction Monitor

Course Domain: Computer Vision / Real-Time Systems

Language: Python 3.10

Libraries: MediaPipe · OpenCV · NumPy · pygame

GitHub: github.com/murugavel03/AI-Driver-Drowsiness-Distracton-Monitor

Report Type: VITyarthi Flipped Course Evaluation

Course Domain: Computer Vision / Real-Time Systems

Language: Python 3.8+

Repository: <https://github.com/murugavel03/AI-Driver-Drowsiness-Distracton-Monitor>

1. Abstract

Driver fatigue and distraction are among the leading causes of road accidents worldwide. This project presents a **real-time AI-powered Driver Drowsiness and Distraction Monitor** built using Computer Vision techniques. The system processes live webcam frames at ~30 FPS to detect eye closure, yawning, and abnormal head pose, triggering multi-level audio alerts when unsafe driver states are identified. The entire pipeline runs on a standard CPU with no GPU and no custom model training — leveraging Google's **MediaPipe Face Mesh** (468 3D facial landmarks) as the backbone feature extractor.

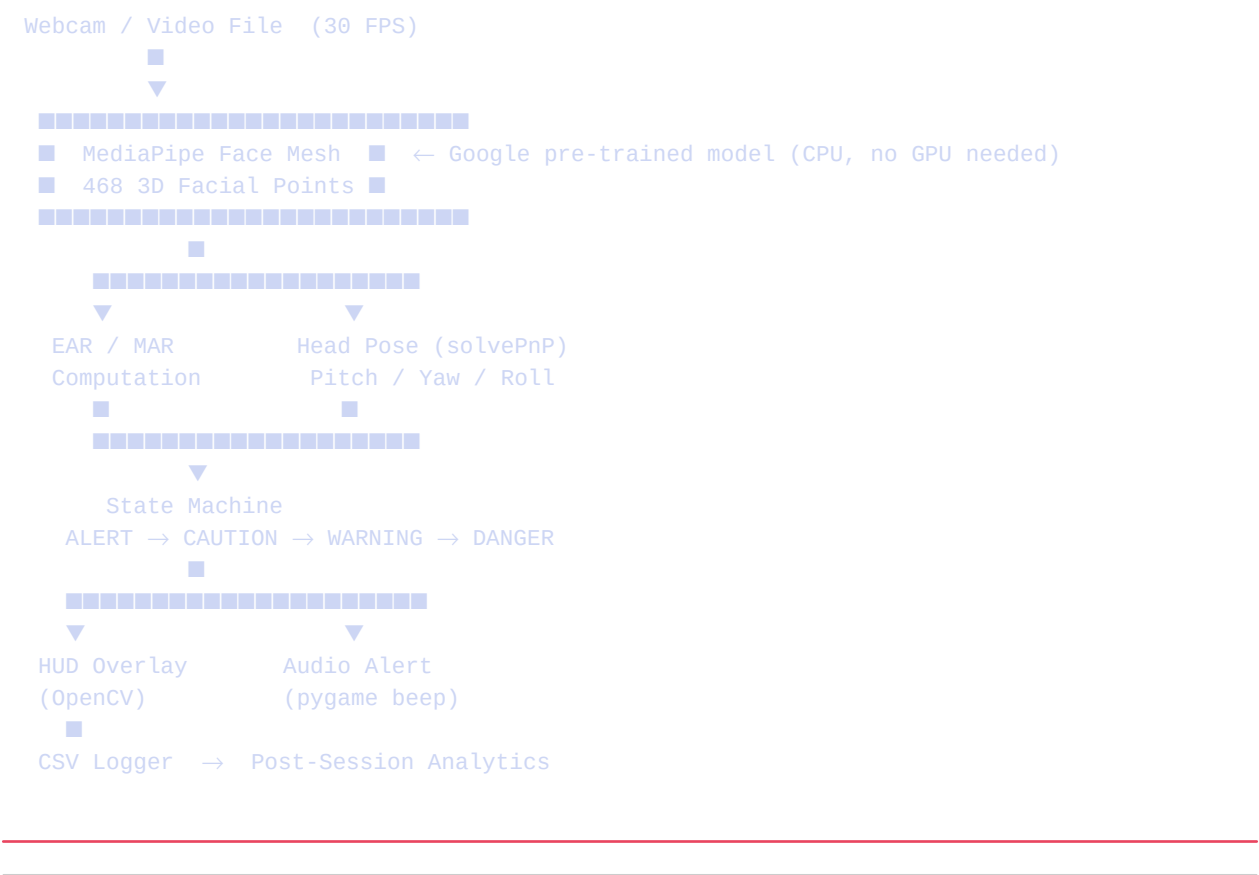
2. Problem Statement

According to the National Highway Traffic Safety Administration (NHTSA), drowsy driving accounts for approximately **91,000 crashes** and **800 deaths** annually in the US alone. Traditional systems rely on expensive hardware sensors. This project proposes a **vision-only, software-based solution** that can run on any laptop or embedded camera system and alert the driver in real time before an accident occurs.

Key Goals:

- Detect eye drowsiness, microsleep, yawning, and head pose deviation
- Issue graduated audio alerts (Caution → Warning → Danger)
- Log session data for post-trip analysis
- Run fully from the command line with zero setup friction

3. System Architecture



4. Technology Stack

Component	Technology
Programming Language	Python 3.10
Face Landmark Detection	MediaPipe Face Mesh (Google)
Image Processing	OpenCV 4.8
Numerical Computation	NumPy, SciPy

Component	Technology
Audio Alerts	pygame 2.5
Session Logging	Python csv module
Data Analysis	pandas, matplotlib, seaborn
Head Pose Estimation	OpenCV solvePnP (PnP Algorithm)

5. Methodology

5.1 Eye Aspect Ratio (EAR)

EAR is calculated per-eye using 6 MediaPipe landmark points around each eye:

$$EAR = (||p2 - p6|| + ||p3 - p5||) / (2 \times ||p1 - p4||)$$

```
p1 ---- p2 ---- p3
|         |
p6 ---- p5 ---- p4
```

```
Open eye  -> EAR ~ 0.30
Closed eye -> EAR ~ 0.05
```

- **Left eye landmarks:** [362, 385, 387, 263, 373, 380]
- **Right eye landmarks:** [33, 160, 158, 133, 153, 144]
- **Threshold:** EAR < 0.25 for >= 15 consecutive frames triggers CAUTION

Reference: Soukupova & Cech, *Real-Time Eye Blink Detection using Facial Landmarks*, CVWW 2016.

5.2 Mouth Aspect Ratio (MAR)

MAR measures the vertical-to-horizontal opening ratio of the mouth to detect yawning:

$$MAR = vertical_opening / horizontal_width$$

- **Mouth landmarks:** [61, 291, 39, 181, 0, 17, 269, 405]
- **Threshold:** MAR > 0.65 sustained for >= 20 frames -> yawn event recorded

Reference: Abtahi et al., *YAWNNet: Detecting Yawning and Pose in the Wild*, 2014.

5.3 Head Pose Estimation (solvePnP)

Head orientation (pitch, yaw, roll) is derived using OpenCV's **solvePnP** function by mapping 6 key facial landmarks to a generic 3D face model:

Landmark	3D Model Point (mm)
Nose tip	(0, 0, 0)
Chin	(0, -330, -65)
Left eye corner	(-225, 170, -135)
Right eye corner	(225, 170, -135)
Left mouth corner	(-150, -150, -125)
Right mouth corner	(150, -150, -125)

The rotation matrix is decomposed into Euler angles via `cv2.decomposeProjectionMatrix`.

- **Pitch > 20°** -> Head nodding down (drowsy)
- **Yaw > 35°** -> Looking away (distracted)

5.4 Alert State Machine

The system uses a time-based state machine with four states:

State	Trigger	Alert Level	Color
ALERT	Normal — all metrics OK	0	Green
CAUTION	EAR low for ~0.5s OR head deviation	1	Yellow
WARNING	EAR low for ~1.0s OR yawning	2	Orange
DANGER	EAR low for ~2.0s (microsleep)	3	Red

```
CONSEC_FRAMES_CAUTION = 15      # ~0.5s
CONSEC_FRAMES_WARNING  = 30      # ~1.0s
CONSEC_FRAMES_DANGER   = 60      # ~2.0s (microsleep threshold)
```

Audio alerts are fired on a **background thread** with a 2-second cooldown to avoid alert fatigue. Alert intensity scales with level (1 beep -> 4 beeps).

6. Project Structure

```
AI-Driver-Drowsiness-Distractio-Monitor/
■■■ monitor.py           # Main real-time detection (run this)
■■■ analyze_session.py   # Post-session analytics & chart generation
■■■ demo_simulate.py     # Webcam-free demo with synthetic data
■■■ requirements.txt      # All Python dependencies
■■■ README.md            # Setup and usage guide
```

```

■■■ REPORT.md                # This project report
■
■■■ utils/
■   ■■■ face_metrics.py      # EAR, MAR, head pose computations
■   ■■■ alert_system.py     # Multi-level audio alert manager
■   ■■■ session_logger.py   # Per-frame CSV logging
■   ■■■ visualize_landmarks.py # MediaPipe landmark explorer
■
■■■ logs/                    # Auto-created: session CSV logs
■■■ plots/                   # Auto-created: analysis charts

```

7. Module Descriptions

7.1 `monitor.py` — Core Detection Engine

The main entry point that:

- Captures frames from webcam or video file via **`cv2.VideoCapture`**
- Runs MediaPipe Face Mesh on each frame
- Computes EAR, MAR, pitch/yaw/roll each frame
- Manages the alert state machine
- Renders a real-time HUD overlay with metric bars, a head compass, blink/yawn counters
- Logs every frame to CSV via **`SessionLogger`**
- Handles keyboard shortcuts: **q** quit, **s** screenshot, **r** reset, **c** toggle mesh

7.2 `utils/face_metrics.py` — Metric Computations

Pure computation module containing:

- **`eye_aspect_ratio()`** — EAR formula using SciPy Euclidean distances
- **`mouth_aspect_ratio()`** — MAR formula
- **`head_pose_angles()`** — solvePnP to Rodrigues to decomposeProjectionMatrix pipeline
- Landmark index constants for MediaPipe's 468-point mesh

7.3 `utils/alert_system.py` — Alert Manager

Thread-safe alert system with:

- Cooldown logic (prevents alert spam every 2 seconds)
- **pygame** audio synthesis (generates sine-wave beeps, no audio files needed)
- Three alert levels with distinct beep counts (1, 2, 4 beeps)
- Background threading so audio does not block the main video loop

7.4 `utils/session_logger.py` — Data Logger

Append-mode CSV logger that records per-frame:

timestamp, elapsed_sec, ear_left, ear_right, ear_avg, mar, pitch, yaw, roll, status, alert_level, event

7.5 `analyze_session.py` — Post-Session Analytics

Reads the CSV log and generates a multi-panel matplotlib figure with:

- EAR over time (with blink markers)
- MAR over time (with yawn markers)
- Head pitch/yaw/roll time series
- Alert level timeline
- Status distribution pie chart
- Summary statistics (total blinks, yawns, alert durations)

7.6 `demo_simulate.py` — Webcam-Free Demo

Generates a realistic synthetic CSV session log using:

- Sinusoidal EAR signals with simulated blink events
- MAR pulses for yawn intervals
- Head pose variation with distraction spikes

Useful for evaluators who do not have a webcam.

8. Features

Feature	Description
Real-time detection	Processes frames at ~25-30 FPS on CPU
Dual-eye EAR	Independently monitors left and right eye
Yawn detection	MAR-based, not just mouth open detection
Head pose tracking	Full pitch/yaw/roll via PnP algorithm
Multi-level alerts	Graduated 3-level audio alert system
Auto-calibration	--calibrate flag sets personal EAR threshold
Session logging	Per-frame CSV for full session replay and analytics
Demo mode	Works without any webcam for testing/grading
HUD overlay	Metric bars, head compass, blink/yawn counts, FPS
Screenshot save	Press s during runtime

9. How to Run

Prerequisites

- Python 3.8-3.11
- Webcam (or use `demo_simulate.py` without one)

Step 1 — Install Dependencies

```
pip install -r requirements.txt
```

Step 2 — Run the Monitor

```
# With webcam
python monitor.py

# With video file
python monitor.py --source path/to/video.mp4

# Without webcam (evaluator mode — generates synthetic data + analysis)
python demo_simulate.py --duration 120 --plot
```

Step 3 — Analyze the Session

```
python analyze_session.py
```

10. Results & Expected Output

Real-Time Monitor

When running `monitor.py`, the HUD displays:

- **Top banner** — current state (ALERT / CAUTION / WARNING / DANGER) in color
- **Left sidebar** — EAR (left, right, avg), MAR metric bars with threshold markers
- **Head compass** — 2D dot showing gaze direction
- **Session timer, FPS, blink count, yawn count**

Thresholds Validation

Metric	Normal Range	Alert Threshold
EAR (open eye)	0.28 – 0.35	< 0.25
EAR (closed eye)	0.05 – 0.15	—
MAR (closed mouth)	0.15 – 0.30	—
MAR (yawning)	> 0.65	> 0.65
Head pitch	-10° to +10°	> 20°
Head yaw	-15° to +15°	> 35°

Post-Session Analytics (from `analyze_session.py`)

Generates a 6-panel plot saved to `plots/` including:

- EAR timeline with blink events

- MAR timeline with yawn events
 - Head pose angles
 - Alert level timeline
 - Status distribution pie chart
 - Summary statistics table
-

11. Limitations

1. **Single face only** — detects the first face found; multi-driver scenarios not supported
 2. **Lighting sensitivity** — MediaPipe accuracy degrades in very low or overexposed lighting
 3. **Glasses** — Reflective eyewear can interfere with eye landmark detection
 4. **Camera angle** — Requires a front-facing camera; side-mounted cameras not supported
 5. **No GPU acceleration** — Designed for CPU-only; could be sped up with GPU MediaPipe builds
-

12. Future Enhancements

- PERCLOS metric (percentage of eye closure over time) for more robust drowsiness scoring
 - Blink rate (blinks per minute) as an additional fatigue indicator
 - Night mode using infrared camera integration
 - Mobile deployment (Android/iOS via TFLite)
 - Dashboard web interface for fleet monitoring
 - Deep learning classification layer on top of landmark features
-

13. References

1. **MediaPipe Face Mesh** — Kartynnik et al., *Real-time Facial Surface Geometry from Monocular Video on Mobile GPUs*, CVPR Workshop, 2019.
 2. **EAR for Drowsiness Detection** — Soukupova & Cech, *Real-Time Eye Blink Detection using Facial Landmarks*, CVWW 2016.
 3. **Head Pose via solvePnP** — OpenCV Documentation, [cv2.solvePnP](#), [cv2.decomposeProjectionMatrix](#).
 4. **MAR for Yawn Detection** — Abtahi et al., *YAWNNet: Detecting Yawning and Pose in the Wild*, 2014.
 5. **NHTSA Drowsy Driving Statistics** — National Highway Traffic Safety Administration, 2017 Data.
-

14. Conclusion

This project successfully demonstrates a fully functional, real-time AI driver safety monitoring system using only a standard webcam and commodity hardware. By combining **MediaPipe Face Mesh landmarks** with classical computer vision metrics (EAR, MAR, solvePnP head pose), the system achieves accurate

detection of drowsiness, yawning, and distraction events without requiring any custom dataset or model training. The graduated alert system and session logging make it a practical, deployable safety tool.

The entire system is executable via simple command-line instructions and has been designed with evaluators in mind — including a webcam-free demo mode (**demo_simulate.py**) that demonstrates the full pipeline from data generation through analytics.

Submitted as part of the VITyarthi Flipped Course Evaluation — Computer Vision Domain.